

Motif Search in GraphEasy

From a pattern in the DSL to a specialised loop nest — a ground-up explanation

GraphEasy (p1GraphEasy-con-AutoTuner)

September 2026

Abstract

This document explains, from the ground up, how *motif search* works in the GraphEasy compiler `p1GraphEasy-con-AutoTuner`: how a pattern such as `a->b; a->c; b->c;` written in a `.graph` program becomes a count of matches, a list of vertex bindings, and one induced subgraph per match. It walks the whole pipeline — grammar, AST, semantic analysis, the two matching backends (a runtime backtracker and a compile-time-specialised IR loop nest), the compile-time pattern analysis, and the runtime — and it records the theory and the design trade-offs behind each decision. Everything labelled *verified* was executed on the tree and the output is transcribed verbatim.

Contents

I	The simple story	2
1	What a motif is	2
2	One example you can check by hand	3
3	What the DSL gives back	4
4	The suite’s pinned example: bi-fan on a 40-arc graph	4
II	The pipeline, stage by stage	4
5	Stage 0 — Grammar	4
6	Stage 1 — AST	5
7	Stage 2 — Semantic analysis	5
8	Stage 3 — Lowering: two backends, one meaning	6
9	Stage 3b — Storage: what is materialised, when, and where	6
10	Stage 4 — Compile-time pattern analysis (MotifPattern)	7
10.1	The required matrix	7
10.2	Automorphisms	8
10.3	Canonicity, reference and fast	8
10.4	Level plans	8

11 Stage 5 — The emitted nest (MotifIRBuilder)	8
12 Stage 6 — The runtime matcher	9
13 Stage 7 — Semantics reference card (all verified)	10
 III Theory, rationale, measurements	 10
14 The algorithmic problem	10
15 What was considered, and why this shape	11
16 Measured behaviour	11
17 Known limits and traps in this tree	12
 IV Using it	 12
18 Commands	12
19 Checklist for a new motif	13
A Code map	13
B Verified evidence log	14

How to read this document

- **Part I** is the simple story: what a motif is, one example you can count by hand, and what the DSL gives you back.
- **Part II** follows one motif program through every compiler stage with code references.
- **Part III** is the theory and the “why this way” discussion, including measured numbers and the traps in the current tree.
- **Part IV** is the practical part: exact commands, env switches, and a checklist for adding a new motif.

Part I

The simple story

1 What a motif is

A **network motif** is a small subgraph pattern that occurs more often in a network than in a suitable random null model (Milo et al., *Network Motifs*, Science 2002). Typical patterns on three or four vertices:

```
(a) 3-chain (b) feed-forward loop (FFL) (c) bi-fan
a -> b -> c a -> b -> c x <-- a
                |_____| | |
                a -> c y <-- b
```

a -> y, b -> x, b -> y

The FFL is the canonical directed triangle with a bypass edge $a \rightarrow b$, $b \rightarrow c$, $a \rightarrow c$; the bi-fan is the four-vertex bipartite pattern $a \rightarrow x$, $a \rightarrow y$, $b \rightarrow x$, $b \rightarrow y$ in which the two “sources” share both “targets”.

Two conventions matter and both are used here:

Induced subgraphs. When you match a pattern against a graph, the subgraph induced by the matched vertices must be *exactly* the pattern: an edge between two matched vertices that is *not* in the pattern makes the match invalid. In the DSL, “not in the pattern” is written by simply not writing the edge — every pair of variables without a motif edge is *required to be absent* (ASTBuilder comment, `ASTBuilder.cpp:16`).

Signs. A pattern edge $a \rightarrow b$ requires an edge of *positive* weight (or an unweighted edge); $a \mid b$ requires a *negative* one. This is what makes signed-network motifs expressible.

2 One example you can check by hand

Take this five-arc directed graph (we will use it throughout; edges are $0 \rightarrow 1$, $1 \rightarrow 2$, $0 \rightarrow 2$, $1 \rightarrow 3$, $0 \rightarrow 3$):

```

      1 --> 2
     ^ \ \
    / \  \ v
0 ----> 3 arcs: 0->1, 1->2, 0->2, 1->3, 0->3

```

Search for an FFL: $a \rightarrow b$; $a \rightarrow c$; $b \rightarrow c$; . A match is an injective map $\{a, b, c\} \rightarrow \{0, 1, 2, 3\}$ such that all three listed arcs exist and all other pairs are absent:

assignment	$a \rightarrow b$	$a \rightarrow c$	$b \rightarrow c$	verdict
$a=0, b=1, c=2$	$0 \rightarrow 1$ ✓	$0 \rightarrow 2$ ✓	$1 \rightarrow 2$ ✓	match
$a=0, b=1, c=3$	$0 \rightarrow 1$ ✓	$0 \rightarrow 3$ ✓	$1 \rightarrow 3$ ✓	match
$a=0, b=2, c=3$	$0 \rightarrow 2$ ✓	$0 \rightarrow 3$ ✓	$2 \rightarrow 3$ ×	no
$a=1, b=2, c=3$	$1 \rightarrow 2$ ✓	$1 \rightarrow 3$ ✓	$2 \rightarrow 3$ ×	no

(Reverse pairs such as $1 \rightarrow 0$ are absent in this graph, so the induced requirement is satisfied for both matches; if the graph contained $2 \rightarrow 0$, the first row would fail because the pair (c, a) would exist although the pattern does not list it.)

In the p1 dialect the program is:

```

graph G { directed: true; edges: file "tiny_edges.txt"; };

motifs M = [G where motif { a->b; a->c; b->c; }];

print "numMotifs";
print numMotifs(M);

for each motif (a,b,c) in M {
  print "binding"; print a; print b; print c;
}

```

Verified output:

```

numMotifs
2
binding
0

```

```

1
2
binding
0
1
3

```

Two matches, and the bindings come out in exactly the two orders we derived by hand.

The induced rule in one experiment. Change the file so that the arc 2→0 also exists. Now the pair (c, a) of the first match is an existing edge that the pattern does not list, so induced matching rejects it — the count becomes 1. This is not a formality: it is the difference between counting *induced* and *non-induced* embeddings, and it is the reason the current tree refuses to find anything in graphs declared with inline edges (see §1).

3 What the DSL gives back

Five constructs use the same matching engine; only the action at a complete assignment differs:

DSL construct	Result
<code>motifs M = [G where motif {...}];</code>	a <i>match collection</i> : the count and the vertex bindings of every match
<code>graphs L = [G where motif {...}];</code>	one <i>induced subgraph</i> per match, so you can run more code on each occurrence
<code>for each motif (a,b,c) in M {...}</code>	iterates the bindings
<code>numMotifs(M), numGraphs(L)</code>	the counts
<code>draw motif of G to "prefix" {...}</code>	renders the matches to an image

Verified on the same five-arc graph (file-backed!):

```

motifs M = [G where motif { a->b; a->c; b->c; }]; numMotifs(M) -> 2
graphs L = [G where motif { a->b; a->c; b->c; }]; numGraphs(L) -> 2
for each graph H in L { numVertices(H) } sum -> 6
for each graph H in L { numEdges(H) } sum -> 2 (see \S\ref{sec:m2})

```

4 The suite’s pinned example: bi-fan on a 40-arc graph

`verify/cases/motif/bifan.graph` is the one motif case in the regression suite:

```

graph G { directed: true; edges: file ".../fixtures/bipartite.txt"; };
motifs BF = [G where motif { a->x; a->y; b->x; b->y; }];
print numMotifs(BF);

```

Verified: the program prints 34, which is exactly the value pinned in `verify/expected/motif.manifest`. This is the “hello world” of the feature: a four-variable, two-source/two-target pattern on a small directed graph.

Part II

The pipeline, stage by stage

5 Stage 0 — Grammar

The motif syntax lives in `Base.g4`:

```
motifMatchesDecl:
    'motifs' ID '=' '[' graphID 'where' 'motif' '{' motifEdge+ '}' ']' ';' ;
graphListDecl:
    'graphs' ID '=' '[' graphID 'where' 'motif' '{' motifEdge+ '}' ']' ';' ;

motifEdge: ID ('->' | '-|') ID ';' ;

graphCondition:
    graphCondition AND graphCondition # GraphLogicalAnd
  | graphCondition OR graphCondition # GraphLogicalOr
  | 'degree' (...) INT # DegreeCondition
  | 'connected' 'with' nodeID # ConnectedCondition
  | 'cycle' # CycleCondition
  | 'motif' '{' motifEdge+ '}' # MotifCondition
  | '(' graphCondition ')' # ParenGraphCondition;

foreachStatement: 'for' 'each' loopTarget 'in' graphID block;
loopTarget: ... | 'motif' '(' ID (',' ID)+ ')' # ForEachMotif | ...;

graphComprehension: ID '=' '[' graphExpr ('where' graphCondition)? ']' ';' ;
```

Read carefully, this fixes several things you might otherwise trip over:

- A motif edge is `ID -> ID` or `ID -| ID` followed by `;`. The left-hand side is a *variable name*, not a vertex id; it is the thing for `each motif (a,b,c)` binds.
- `motifs` and `graphs` are separate statement forms with a fixed `where motif` keyword order.
- The comprehension rule's left-hand side is a bare `ID`, so in *p1* you write `F = [G where ...]`; and `not graph F = [G where ...]`; (`graph` starts a graph *declaration*). This was verified: `graph F = [G]`; fails to parse with “extraneous input ‘[’” at the `[`.

6 Stage 1 — AST

`ASTBuilder.cpp:16-26` turns one `motifEdge` into

```
struct MotifEdgeSpec { std::string source, target; MotifEdgeSign sign; };
enum class MotifEdgeSign { Positive = 1, Negative = -1 }; // ASTNode.h:45
```

The declaration becomes a `MotifMatchesDeclNode` carrying the graph name, the result name, the edge list and the *variable order* (`ASTNode.h:1196`), and `for each motif (...)` becomes `ForEachStatementNode` with `targetType == Motif` and `motifVars`. A motif inside a comprehension condition becomes a `GraphConditionNode` with `op == GraphConditionOp::Motif` (`ASTBuilder.cpp:1349`).

7 Stage 2 — Semantic analysis

`SemanticAnalyzer.cpp:980` (`analyzeMotifMatchesDecl`) enforces the contract that the rest of the compiler relies on:

- the graph must exist, and must be declared `directed: true` — motif matching over an undirected graph cannot express the induced-absence rules meaningfully;
- at least one edge and at least two variables;
- at most **six** variables (`kMotifMaxVars`); and
- no self edges (`a->a`) and no duplicate edges for the same ordered pair.

`numMotifs` is typed against a `MotifMatches` symbol (`SemanticAnalyzer.cpp:206`), and for each motif (`a,b,c`) checks that the binding count matches the collection’s variable count (`SemanticAnalyzer.cpp:682`).

8 Stage 3 — Lowering: two backends, one meaning

`IRGenVisitor.cpp` has two ways to answer “which vertices match this pattern”:

1. **Runtime backend** (default). `visitMotifMatchesDecl` (`IRGenVisitor.cpp:1390`) marshals the motif edges into three `i32` arrays (source var, target var, sign) and calls `motif_find_matches_runtime_ptr, col_idx, weights, src, dst, sign, edge_count, var_count`). The result is a heap struct `MotifMatchesRuntime {count, var_count, bindings, capacity}`. This backend produces *bindings*, so it serves every construct.
2. **IR backend**, selected with `GRAPHEASY_MOTIF_BACKEND=ir` (`IRGenVisitor.cpp:1318`). `emitMotifMatchesIR` (`IRGenVisitor.cpp:1361`) builds a `MotifPattern` from the same edge list, then asks `MotifIRBuilder` to emit a specialised loop nest and to count complete assignments. It wraps the count in a stack `MotifMatches` with `bindings = NULL`.

Two consequences worth knowing before you rely on them:

- the IR backend currently serves *only* motifs `X = [...]`, and only its **count**. for each motif, graphs `L = [...]` and `draw` always use the runtime backend;
- `GRAPHEASY_MOTIF_ADJDRIVE=1` additionally switches the emitted nest to enumerate CSR rows instead of scanning $0..n - 1$ at every level (§11, §16).

9 Stage 3b — Storage: what is materialised, when, and where

Both backends speak one data layout, deliberately: the runtime heap struct

```
/* runtime.c:1135 -- heap, one per 'motifs' / 'graphs' declaration */
typedef struct MotifMatchesRuntime {
    int32_t count; /* matches found */
    int32_t var_count; /* k, the row width */
    int32_t *bindings; /* flat row-major array, k int32 per match */
    int32_t capacity; /* allocated rows (>= count) */
} MotifMatchesRuntime;
```

and the compiler’s IR type `struct.MotifMatches` are the same 24-byte `{i32,i32,i32*,i32}` layout, so “a value produced by the runtime path and one produced by the IR path stay interchangeable” (`IRGenVisitor.h:40-47`; `IRGenVisitor.cpp:1377-1387`).

The bindings array. Match *i* occupies `bindings[i*k .. i*k+k-1]`. It grows by doubling (capacity 16, then $\times 2$) through `realloc + memcpy` of one row at every append (`motif_matches_append_runtime.c:1192`); appends happen at the leaves of the backtracking search, i.e. *as matches are discovered*. The array is never freed: it lives for the rest of the program, costing $4k \cdot \text{count}$ bytes of rows (with up to a factor-two slack from the last doubling). Declaring `motifs/graphs` is therefore *eager* — `numMotifs` does not avoid the search; it reads `count`.

What each construct materialises.

construct	storage	when
<code>motifs M = [...]; (runtime)</code>	heap struct + all bindings rows	eagerly, at the declaration
<code>motifs M = [...]; (IR backend)</code>	a stack struct with <code>count</code> only; <code>bindings = NULL</code>	count computed at the declaration
<code>numMotifs(M), numGraphs(L)</code>	nothing new; reads <code>count</code>	—
<code>for each motif (a,b,c) in M</code>	nothing new; reads <code>bindings[i*k+j]</code>	per iteration
<code>graphs L = [...];</code>	the same heap struct (bindings)	eagerly, at the declaration
<code>for each graph H in L</code>	H = a fresh induced subgraph per iteration via <code>graph_from_match_runtime</code> (deep CSR copy, never freed)	lazily, one per iteration
<code>draw motif(s) ...</code>	nothing: a private streaming back-tracker renders each match as found (<code>draw_motif_backtrack_runtime</code> , <code>runtime.c:1522</code>)	streaming

Count-only vs listing. *Listing* means reading the `bindings` array: for each motif, for each graph, and the streaming drawing path. *Count-only* means reading `count`: `numMotifs`, `numGraphs`, and the whole IR backend, which never stores individual matches. Mixing them is the trap: an IR-built collection has `count > 0` and `bindings = NULL`, so iterating it reads through a null pointer.

motifs vs graphs, precisely. Both run the same matcher and store the same binding rows; they differ only in the loop variable and the per-iteration materialisation. `motifs` gives the variable bindings (`for each motif (a,b,c) in M`, names bound *positionally* in first-appearance order); `graphs` gives one induced subgraph per match (`for each graph H in L`, H usable as an ordinary graph, rebuilt on the fly). “F = [G where motif {...}];” is the comprehension form and is a third thing again — currently the condition is dropped there entirely (§17).

10 Stage 4 — Compile-time pattern analysis (MotifPattern)

`MotifPattern.h` is deliberately free of LLVM dependencies so that it can be unit-tested against a transcription of the runtime semantics. Given the edge list and variable names it produces four things.

10.1 The required matrix

Mirroring the runtime exactly (`runtime.c:1115-1121`):

```
enum class ReqKind : int8_t { Absent = 0, Positive = 1, Negative = -1 };
// build(): for each edge (src,dst,sign):
// self edges are ignored; a later edge overwrites an earlier one;
// required[src][dst] = sign<0 ? Negative : Positive;
// everything else stays Absent.
```

Absent is not “don’t care”. The matcher requires `actual == required` in *both* directions for every pair of bound variables, so an absent entry means “this edge must not be present in either sign” — the induced rule.

10.2 Automorphisms

A permutation π of the variables preserves the pattern if `required[ $\pi(i)$ ][ $\pi(j)$ ] == required[i][j]` for all i, j , i.e. the whole matrix, signs and absences included, is invariant. All permutations are enumerated ($k \leq 6 \Rightarrow$ at most 720), which is free at compile time (`MotifPattern.cpp:74`).

The automorphism group is what makes counting well-defined: the same set of vertices is reachable by several variable assignments, and we want one representative per orbit.

10.3 Canonicity, reference and fast

The runtime decides “is this assignment the representative?” by the honest $O(k!)$ test: reject the assignment if *any* automorphic image is lexicographically smaller (`runtime.c:1238-1266`). The emitter uses an equivalent reduction, derived in `MotifPattern.h`:

For every non-identity automorphism π , let $p = \min\{i : \pi(i) \neq i\}$. For *injective* assignments the first index where the permuted vector can differ is exactly p , so “some automorphic image is smaller” collapses to a single comparison $A[p] < A[\pi(p)]$, and $p < \pi(p)$ always holds. Hence the whole canonicity test is a conjunction of *lower bounds*: for each such pair, require

$$\text{assignment}[p] < \text{assignment}[\pi(p)].$$

Only the fast form is used by the emitter; the reference form is kept for tests.

10.4 Level plans

For level j (the j -th variable in binding order) the plan says where candidates come from and what they must satisfy:

```
enum class LevelSource { FullScan, OutRow, InRow };
struct LevelCheck { int prev; ReqKind forward, backward; };
struct LevelPlan { LevelSource source; int driverVar;
                  std::vector<LevelCheck> checks;
                  std::vector<int> lowerBounds; };
```

`checks` lists every already-bound variable with the two required kinds (`required[j][prev]` and `required[prev][j]`). The driver for adjacency-driven enumeration is the *most recently bound* variable that has a required edge to/from j : if `required[prev][j]` is present the candidate is an out-neighbour of $A[prev]$ (`OutRow`); otherwise, if `required[j][prev]` is present, an in-neighbour (`InRow`). The heuristic is cache-conscious (“most constrained, most recently touched”), and it never skips a probe: an edge that exists with the wrong sign still fails the kind test.

11 Stage 5 — The emitted nest (`MotifIRBuilder`)

`emitNest` unrolls exactly k levels. For each level it creates the blocks `cond/body/inc/done` and generates this shape (pseudo-IR):

```
level j:
  cond: idx = phi(base, next); if (idx < end) body else done
  body: candidate = loadCandidate(idx) ; full scan: idx
        ; OutRow : col_idx[idx]
```

```

guards: canonical lower bounds (cheapest, prunes whole subtree)
        candidate != A[p] for all p < j (injectivity)
        kind(candidate, A[p]) == required[j][p] (forward)
        kind(A[p], candidate) == required[p][j] (backward)
recurse into level j+1 with candidate pushed
inc: next = idx + 1
done: -> exit target (which is the increment block of the parent)

```

Three details carry the correctness argument and are quoted from the code:

- **the probe** is a single emitted function `__ge_motif_edge_kind(row_ptr, col_idx, weights, n, u, v)` that reproduces `graph_directed_edge_kind_runtime` exactly: scan row u for v , first hit wins; `weights == NULL` means unweighted ($\rightarrow 1$); else the sign of the weight decides ($> 0 \rightarrow 1$, $< 0 \rightarrow -1$, $= 0 \rightarrow 2$). Emitting it once and letting `-O3` inline it keeps the semantics “obviously identical to the C version” (`MotifIRBuilder.cpp:12-22, 133`);
- **canonicity is tested first** because a failing lower bound prunes the entire subtree below the candidate (`MotifIRBuilder.cpp:196`);
- **injectivity is k compares**, not a `used[]` array: at $k \leq 6$ that is cheaper than `calloc(n)` plus a zero fill (`MotifIRBuilder.cpp:208`).

The leaf action is a callback. The tree instantiates it only as `emitCountOnly` (increment an `i64` counter); the header documents the general `LeafEmitter` used by “all five DSL constructs”, which is the intended extension point for lowering for each motif to the IR backend.

12 Stage 6 — The runtime matcher

For reference, the interpreter that the IR nest replaces. In `runtime.c:1271-1315` (`motif_collect_backtrack_`

```

if (pos == var_count) {
    if (!motif_assignment_is_canonical_runtime(...)) return 1; // dedupe
    return motif_matches_append_runtime(matches, assignment); // bindings
}
for (candidate = 0; candidate < n; ++candidate) {
    if (used[candidate]) continue;
    for (prev = 0; prev < pos; ++prev)
        if (kind(candidate, A[prev]) != required[pos][prev] ||
            kind(A[prev], candidate) != required[prev][pos]) goto next;
    A[pos] = candidate; used[candidate] = 1;
    recurse(pos+1);
    used[candidate] = 0;
}

```

- enumerate candidates $0..n-1$, filter by `used[]`, then by both directions of the required matrix for every earlier variable;
- at a complete assignment, apply the canonicity test and append the binding vector (`motif_matches_append_runtime`, growth by doubling);
- for graphs `L = [...]`, each binding is turned into an induced subgraph by `graph_from_match_runtime` (`runtime.c:1155`): mark the matched vertices in a roaring bitmap and rebuild a CSR with only the arcs whose both endpoints are marked (`build_subgraph_from_bitmap_runtime`, `runtime.c:759`).

- `graph_motif_filter_runtime` (`runtime.c:1078`) is the comprehension-filter variant: it collects the *union* of matched vertices and returns the induced subgraph on that set. In this tree it has no caller — a comprehension written `F = [G where motif {...}];` parses and runs, but the condition is silently dropped (§17).

13 Stage 7 — Semantics reference card (all verified)

Graph state for pair (u, v)	kind	satisfies
no edge	0	Absent only
edge, unweighted	1	Positive
edge, weight > 0	1	Positive
edge, weight < 0	-1	Negative
edge, weight $= 0$	2	<i>nothing</i> — both present and absent requirements fail

Experiments behind the table (small weighted file graph $0 \rightarrow 1$ (5), $1 \rightarrow 2$ (5), $0 \rightarrow 2$ (w); every run verified):

weight of the chord $0 \rightarrow 2$	pattern	numMotifs
-3	<code>a->b; b->c; a->c;</code>	0 (chord is not positive)
-3	<code>a->b; b->c; a- c;</code>	1 (chord is negative: <code>- </code> is a <i>present negative</i> edge)
-3	<code>a->b; b->c; c- a;</code>	0 ($2 \rightarrow 0$ does not exist; use omission for “absent”)
0	<code>a->b; b->c; a->c;</code>	0
0	<code>a->b; b->c; a- c;</code>	0
0	<code>a->b; b->c;</code>	0 (chord must be <i>absent</i> , but a zero-weight edge is present)

The last row is the subtle one: a zero-weight edge *poisons* the pair, so it prevents an induced match even when the pattern never mentions the pair. `MotifIRBuilder.cpp:20` states exactly this (“matches no **required** value, so it poisons the pair for both present- and absent-requirements”), and the experiment confirms it.

Part III

Theory, rationale, measurements

14 The algorithmic problem

Subgraph isomorphism is NP-hard in general; motif counting is normally done in the *small-k* regime ($k \leq 6$ here). The three things that make this implementation practical:

1. **fixed small k** : the nest is unrolled, the pattern is a compile-time constant, and no general isomorphism machinery is needed;
2. **induced semantics** turns “which edges are forbidden” into data, not into a special case: the **required** matrix is total;
3. **canonical representatives** convert listing into counting by orbits of the automorphism group, so each vertex set is reported once, and the fast form turns the test into a handful of integer compares that are evaluated *while descending* (so they prune).

15 What was considered, and why this shape

Why not keep the runtime backtracker only? It works and is the reference, but it is written against data: it marshals arrays, rebuilds the `required` matrix at run time, scans $0..n - 1$ at every level, keeps a `used[n]` array, and re-runs an $O(k!)$ canonicity search per complete assignment. All of that is known when the compiler runs. `MotifPattern.h` states the motivation explicitly: “Everything in this file is the part of that work that can be done once, at compile time, so the emitted IR carries constants instead of an interpreter.”

Why not a general subgraph-isomorphism library (VF2/RI/CFL)? Those add feasibility pruning for large patterns and large graphs, at the cost of a general engine. Here $k \leq 6$ with induced constraints and a fixed pattern: the dominating costs (scanning all n per level, per-assignment canonicity) are removed by specialisation, and the remaining algorithm is simple enough to verify against a 40-line transcription.

Why automorphism orbits rather than, say, sorting the binding by vertex id? Sorting by id would pick one representative per *vertex set*, which is not what a pattern query means when the pattern itself has a non-trivial automorphism group (for example, a pattern with two interchangeable leaves must not report the same embedding twice, but must still report mirror images that the pattern distinguishes). Orbit canonicity is the minimal exact test.

Why the fast canonicity reduction is safe Because bindings are injective: see the boxed argument in §4.3 (first moved index p , and $p < \pi(p)$ is forced).

Why the “first hit wins” probe? To be *identical* to the runtime, which matters more here than micro-optimisation: parallel edges with different signs resolve to the first one in CSR order, and the zero-weight rule has a single definition. The IR backend deliberately does not “improve” this; `-O3` inlines the probe and folds the constant comparison.

Why induced, not monomorphism? Induced subgraph counts are the standard network-motif convention (Milo et al.), and they are the reason the matcher must check *both* directions of every pair, including absences (`MotifIRBuilder.cpp:216`).

16 Measured behaviour

Random directed graph, $n = 2000$, $m = 6000$ arcs (seed 7, generated for this document), FFL pattern `a->b; a->c; b->c;`, same program compiled three ways; counts identical, three timed runs each (wall clock, one process per run):

backend	count	3 runs (s)	per run (s)
runtime (default)	26	0.837	0.279
IR nest (<code>GRAPHEASY_MOTIF_BACKEND=ir</code>)	26	0.658	0.219
IR nest + adjacency driven (<code>...=ir, GRAPHEASY_MOTIF_ADJDRIVE=1</code>)	26	0.049	0.016

The adjacency-driven nest is $\approx 17\times$ faster than the runtime interpreter here, and the agreement of all three counts (26) is itself a differential test: three independent code paths, one answer.

The suite’s pinned case is the bi-fan on a 40-arc directed fixture: `numMotifs = 34` (verified), matching `verify/expected/motif.manifest`.

17 Known limits and traps in this tree

1. **Inline-edge graphs are stored undirected.** A graph declared as `graph G { directed: true; nodes: 0,1,2,3; edges: 0->1, ...; }` answers “neighbours of 1” with `{0,2,3}` — the reverse arc is materialised. Therefore every induced pattern that omits the reverse pair fails, and `numMotifs` is 0; the identical topology loaded from a file gives the correct 2. *Workaround (verified): declare graph edges with `edges: file "..."` for motif search.* The file loader preserves arc directions, as does `edge` iteration over file graphs.
2. **where motif in a comprehension is silently ignored.** `F = [G where motif { a->b; a->c; b->c; }];` compiles and runs, but `numVertices(F)` equals the unfiltered graph (verified: $n = 4$ with and without the clause), while `F = [G where degree > 100];` filters correctly ($n = 0$). The reason is visible in `IRGenVisitor.cpp:1660-1716`: the postfix condition emitter has cases for `Connected`, `Cycle`, `Degree`, `And`, `Or` — and no case (and no default) for `GraphConditionOp::Motif`, so the motif constraint contributes no filter token. Use `motifs/graphs` declarations instead.
3. **graph F = [...] is the p2 spelling.** In `p1` a comprehension is a bare `ID = [...]`, and the `graph` keyword before it is a parse error (verified).
4. **The IR backend is count-only** and only for motifs declarations. `for each motif` over an IR-built collection would read a null `bindings` pointer — do not mix them.
5. **InRow enumeration needs a private transpose**, which is not wired up in the emitted nest: `MotifIRBuilder.cpp:168` asserts if a level plan asks for `InRow` under adjacency-driven mode. Patterns whose only adjacency anchor points backwards therefore need the full-scan fallback.
6. **numEdges reports undirected edges:** the lowering emits $m / 2$ (`IRGenVisitor.cpp:4916-4927`). On a directed graph with 5 arcs it prints 2, and each 3-arc match graph prints 1.
7. **Duplicate motif edges are rejected** at semantic level, and the pattern builder also implements the runtime’s *later-wins* rule, so the two definitions agree even for inputs that slip past the checker.

Part IV

Using it

18 Commands

```
cd p1GraphEasy-con-AutoTuner

# compile + run + time, runtime backend (default)
rm -f final_program
GRAPH_FILE=my_motif.graph bash ./03_run.sh

# same program, IR backend, full scan levels
GRAPH_FILE=my_motif.graph GRAPHEASY_MOTIF_BACKEND=ir bash ./03_run.sh

# IR backend with adjacency-driven enumeration
GRAPH_FILE=my_motif.graph GRAPHEASY_MOTIF_BACKEND=ir \
  GRAPHEASY_MOTIF_ADJDRIVE=1 bash ./03_run.sh

# the suite's motif check
cd ../verify && bash run.sh motif # motif/bifan (=34)
```

Useful environment switches: `GRAPHEASY_MOTIF_BACKEND=ir|runtime`, `GRAPHEASY_MOTIF_ADJDRIVE=0|1`.

19 Checklist for a new motif

1. Write the pattern as motif edges over fresh variable names; remember the induced rule — every pair you do not mention must be absent in the data.
2. Declare the graph with `directed: true` and load it from a file (§1).
3. Use `motifs` to count or to iterate bindings; use `graphs` to run further code on each occurrence; `draw motif` for pictures.
4. If you need speed on a large graph, compile with `GRAPHEASY_MOTIF_BACKEND=ir GRAPHEASY_MOTIF_ADJDRIVE=0` — but only where a count is enough (bindings are not produced by that path).
5. Sanity-check small instances by hand (§2): the count of a pattern you can enumerate is the fastest way to catch a semantic misunderstanding — signs, zero weights, or the induced rule.

A Code map

What	Where
Grammar: <code>motifs/graphs/motifEdge/where motif</code>	<code>Base.g4:135-171</code>
Grammar: comprehension	<code>Base.g4:126-128</code>
AST: edge spec and signs	<code>ASTBuilder.cpp:16-26</code> , <code>ASTNode.h:41-51</code>
AST: declaration node	<code>ASTNode.h:1196</code> , <code>ASTBuilder.cpp:230</code>
Semantics: limits and checks	<code>SemanticAnalyzer.cpp:980-1010</code> , 206, 682
Lowering: backend switch	<code>IRGenVisitor.cpp:1318-1332</code>
Lowering: runtime path	<code>IRGenVisitor.cpp:1390-1449</code>
Lowering: IR path (count)	<code>IRGenVisitor.cpp:1361-1388</code>
Lowering: <code>graphs</code> path	<code>IRGenVisitor.cpp:1460-1530</code> , <code>runtime.c:1155</code>
Lowering: <code>for each motif</code>	<code>IRGenVisitor.cpp:4256</code>
Lowering: <code>numMotifs</code> , <code>numEdges</code>	<code>IRGenVisitor.cpp:4886</code> , 4916
Lowering: <code>draw motif</code>	<code>IRGenVisitor.cpp:1197-1290</code>
Condition emitter (missing <code>Motif</code> case)	<code>IRGenVisitor.cpp:1660-1716</code>
Pattern: required matrix, automorphisms, bounds, plans	<code>MotifPattern.cpp:23-168</code>
Nest: probe, ranges, filters, levels	<code>MotifIRBuilder.cpp:23-325</code>
Runtime: probe, backtrack, canonical, collect	<code>runtime.c:1013-1357</code>
Storage: match struct, growth, eager collection	<code>runtime.c:1135-1210</code>
Storage: IR layout of the same struct	<code>IRGenVisitor.h:40-47</code>
Storage: <code>for each motif / for each graph</code> reads	<code>IRGenVisitor.cpp:4300-4390</code>
Storage: streaming draw	<code>runtime.c:1522</code> , 1591
Runtime: induced subgraph	<code>runtime.c:759-905</code> , 1155-1190
Suite	<code>verify/cases/motif/bifan.graph</code> , <code>verify/expected/motif.manifest</code>

B Verified evidence log

All outputs below were produced on 2026-09-24 with the compiler built from the tree; commands are as in §“Commands”.

1. Hand example (file graph 0->1,1->2,0->2,1->3,0->3): FFL count 2, bindings (0,1,2) and (0,1,3).
2. `graphs` collection: `numGraphs` 2, vertex total 6, `numEdges` total 2 (m/2 convention).
3. Bi-fan fixture: `numMotifs` 34 (`motif.manifest` pins 34).
4. Signed graph 0->1(5), 1->2(5), 0->2(-3): triangle pattern 0, a-|c variant 1, c-|a variant 0.
5. Zero-weight chord 0->2(0): all three patterns 0 (poison).
6. Inline vs file declaration: neighbours of 1 = {0,2,3} vs {2,3}; FFL 0 vs 2.
7. Comprehension `where motif`: same result as no filter (dropped); `where degree > 100`: $n = 0$ (filter works).
8. `graph F = [G];`: parse error at [`.`
9. Timing on $n=2000, m=6000$ (FFL): 0.279 / 0.219 / 0.016 s per run for runtime / IR / IR+adjacency, count 26 in all three.